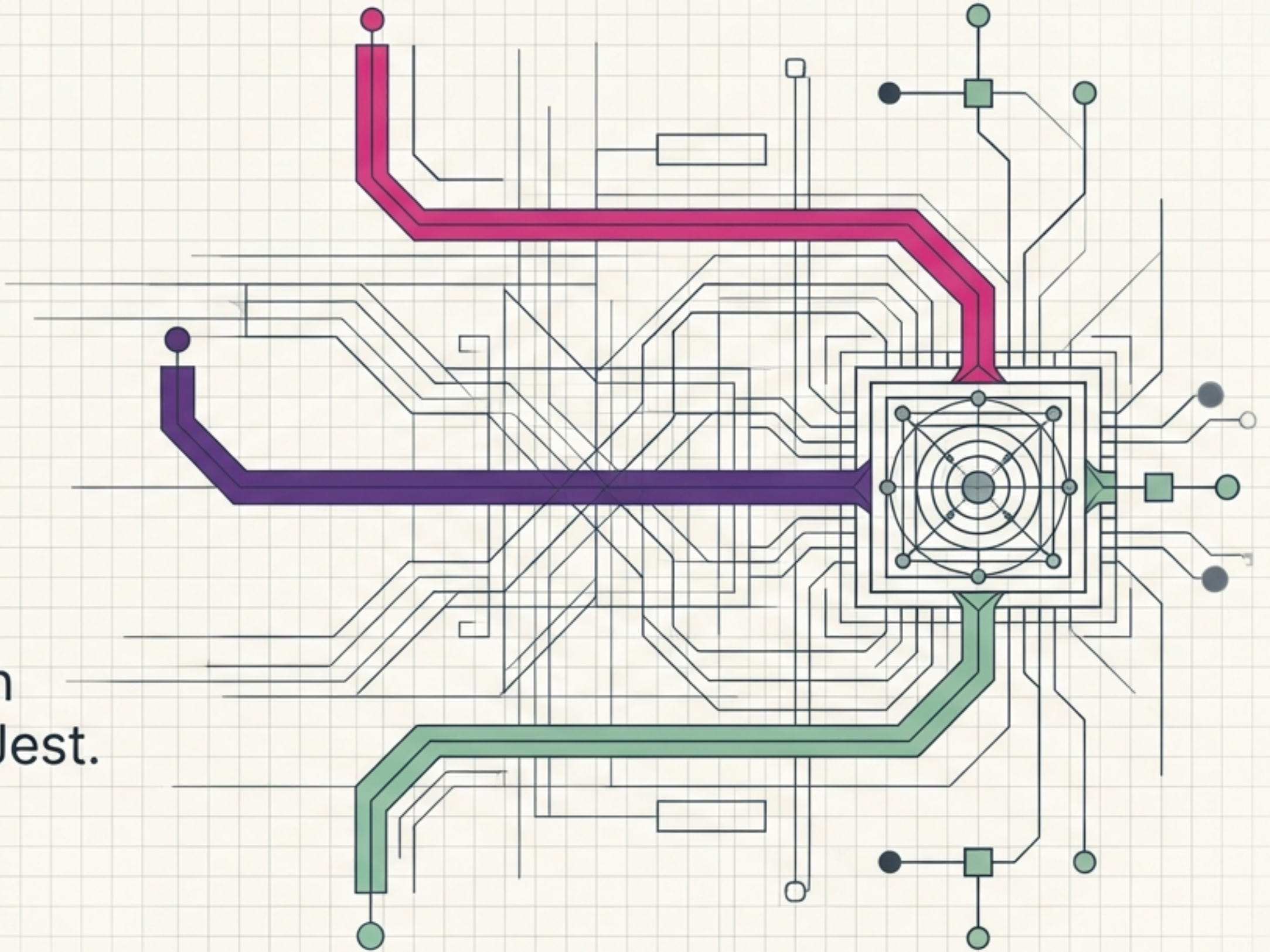
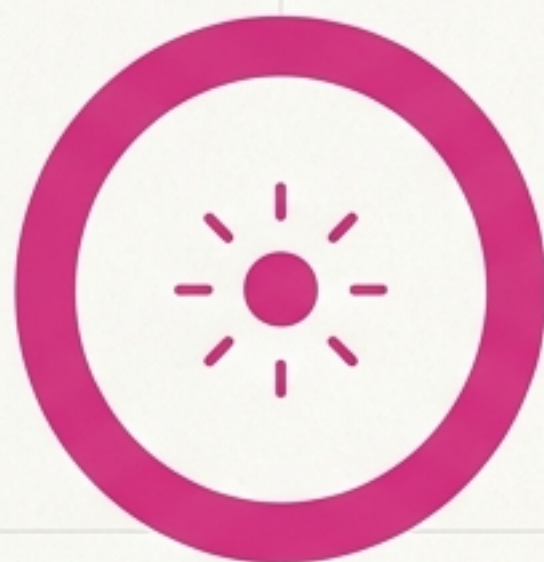


Evolución Reactiva: Del Dato al Ecosistema

Arquitectura y validación en
Angular con RxJS, NgRx y Jest.





RxJS (Lo Micro)

El ADN de la reactividad.
Controlando el flujo asíncrono
y el estado local básico a nivel
de componente.



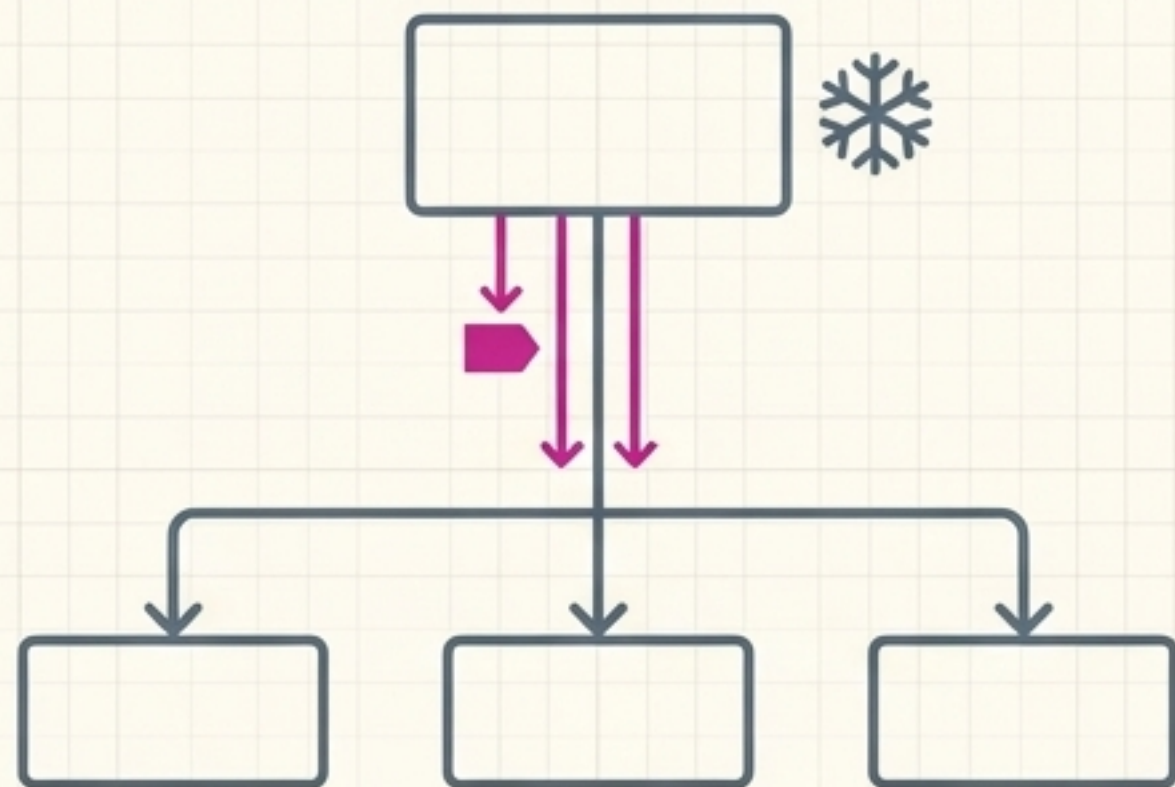
NgRx (Lo Macro)

El motor de estado global.
Orquestando múltiples
fuentes de datos con un flujo
unidireccional e inmutable.



Jest (La Validación)

El control de calidad.
Asegurando que las partículas
y el sistema funcionen sin
fricciones.

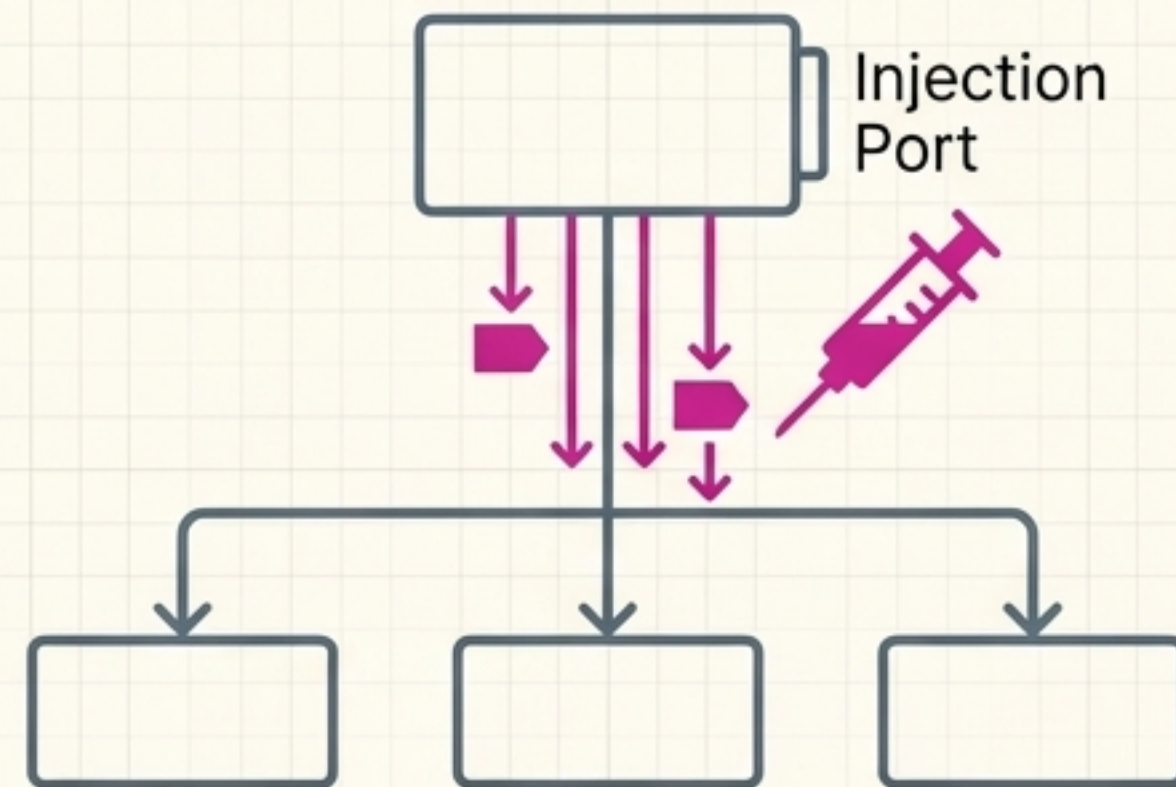


Observable (El Cauce Pasivo)

Frío por defecto: Roboto Mono
No hace nada hasta que un consumidor se suscribe.

Solo lectura: Roboto Mono
Secuencia asíncrona que no permite empujar valores externamente.

Uso ideal: Roboto Mono
Peticiones HTTP y eventos del DOM.



Subject (El Transmisor Activo)

Bidireccional: Roboto Mono
Actúa simultáneamente como Observable y como Emisor.

Inyección Manual: Roboto Mono
Permite empujar nuevos valores al flujo utilizando el método `.next()`.

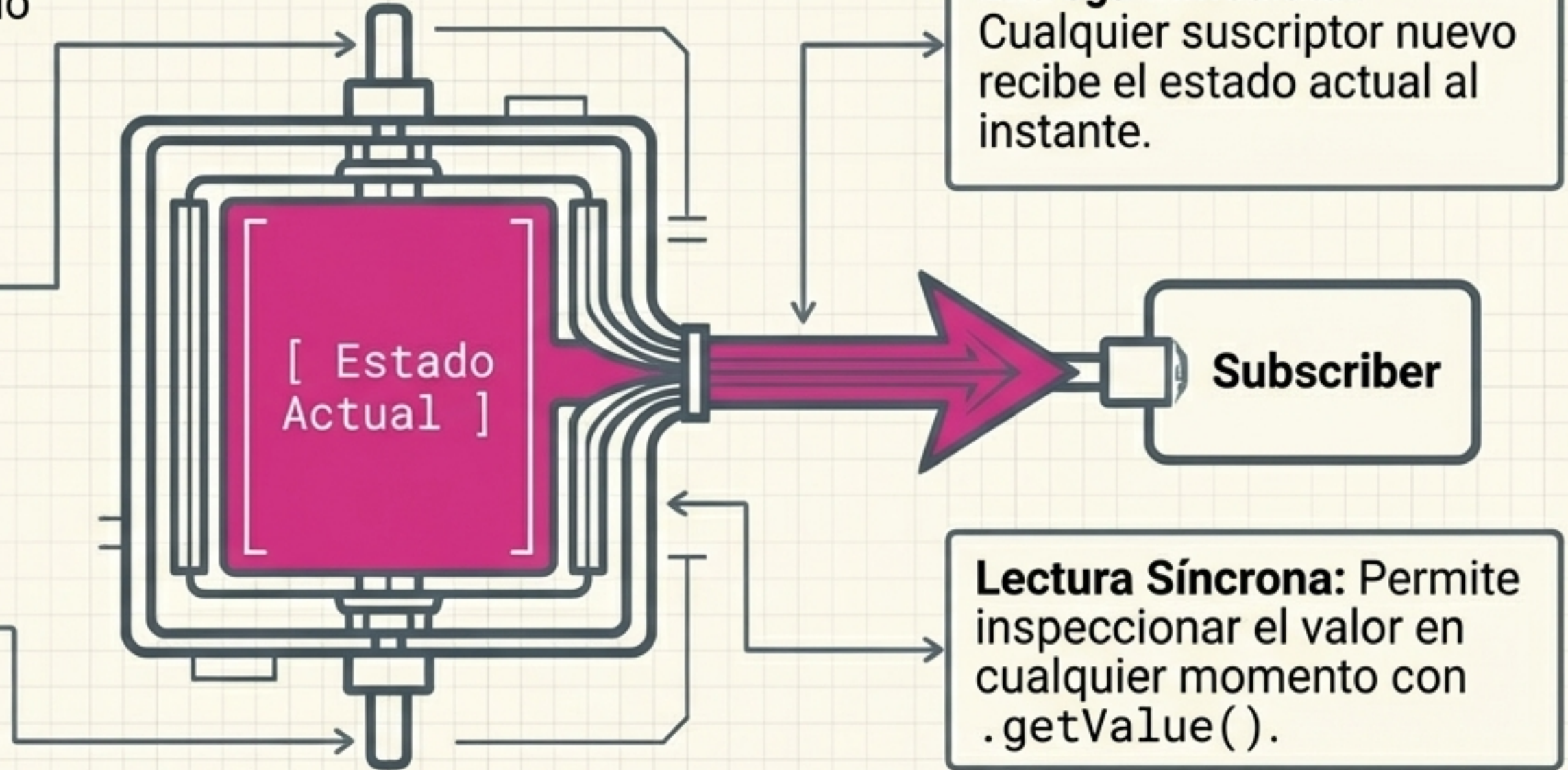
Uso ideal: Roboto Mono
Emisión de eventos manuales dentro de la aplicación.

BehaviorSubject: El Núcleo del Estado Local

A diferencia de un Subject normal, el **BehaviorSubject** tiene memoria. Es la herramienta ideal para compartir estado entre componentes vinculados.

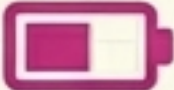
Valor Inicial: Requiere obligatoriamente un valor al instanciarse.

Memoria Intacta: Guarda siempre el último valor emitido.



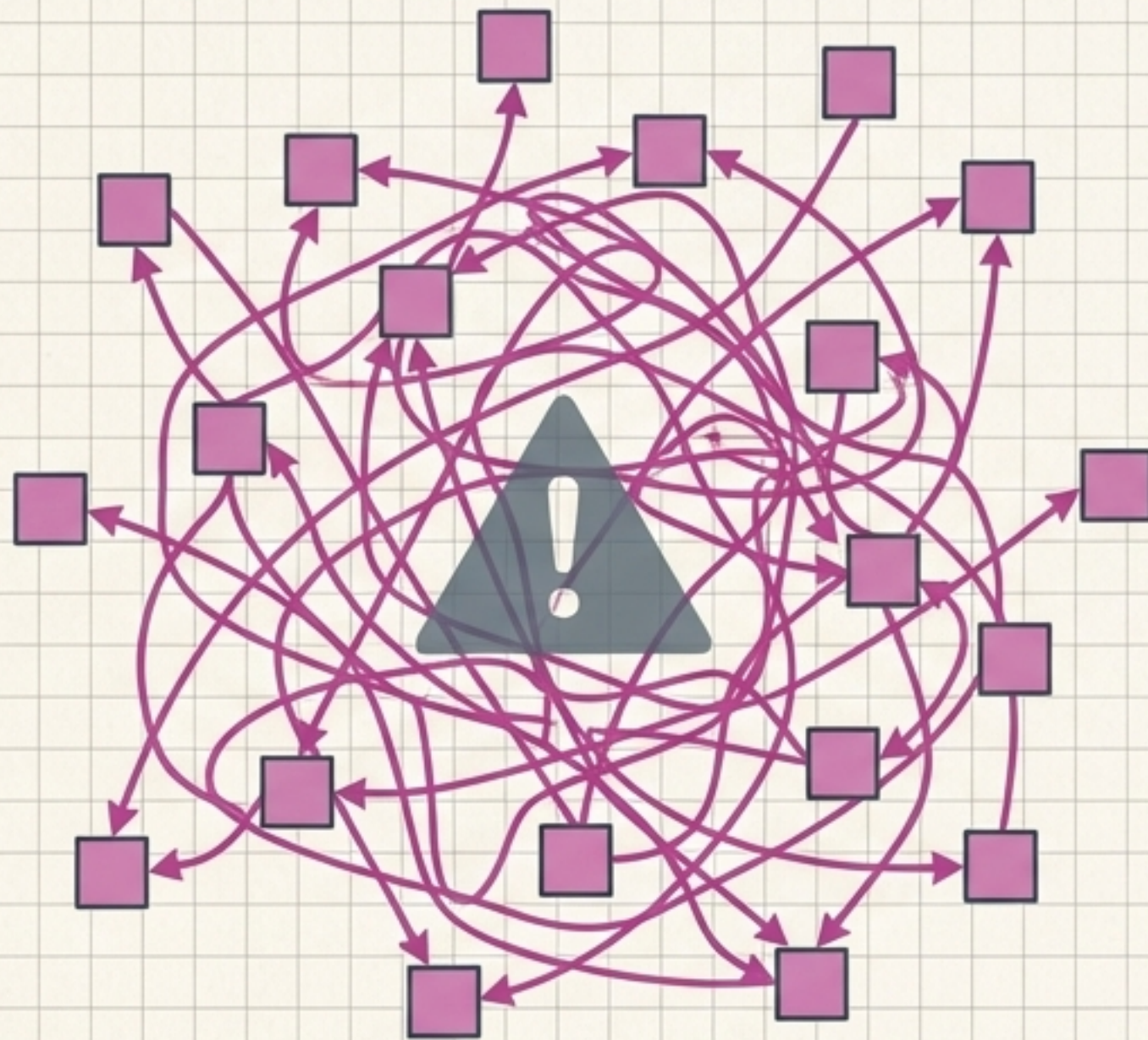
Regla de Oro Arquitectónica: Al exponer un BehaviorSubject desde un servicio, hazlo siempre a través de `.asObservable()`. Esto previene que componentes externos muten el estado llamando a `.next()` directamente.

La Matriz de Mutación RxJS

Característica	Observable	Subject	BehaviorSubject
Valor inicial requerido	-	-	 Sí
Emite último valor al suscribirse	-	-	 Sí
Permite inyectar con .next()	-	 Sí	 Sí
Lectura directa con .getValue()	-	-	 Sí
Uso Ideal	HTTP, Streams pasivos	Eventos manuales	Estado compartido

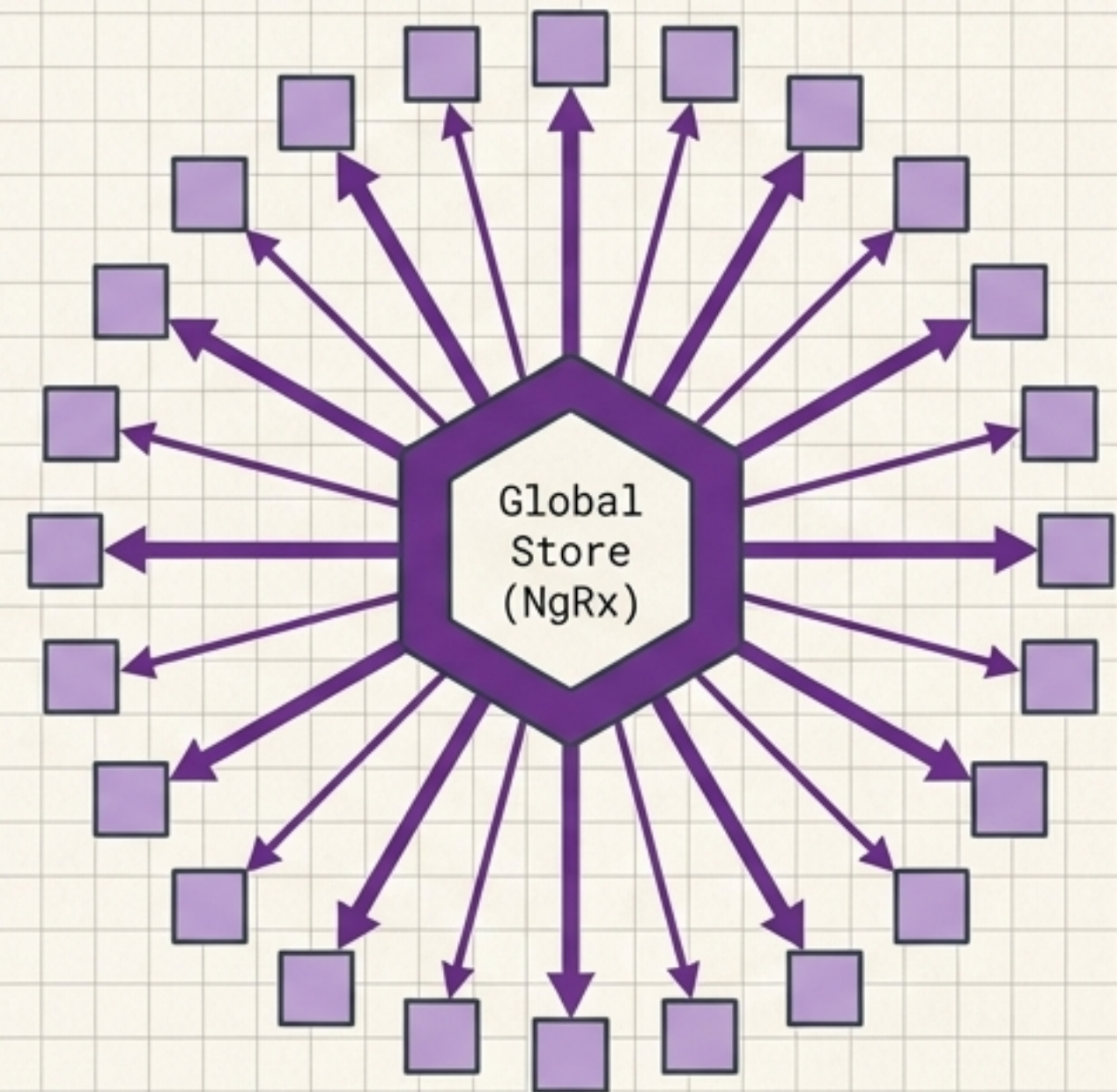
El Límite del Estado Local: Cuándo el Caos Escala

El Problema



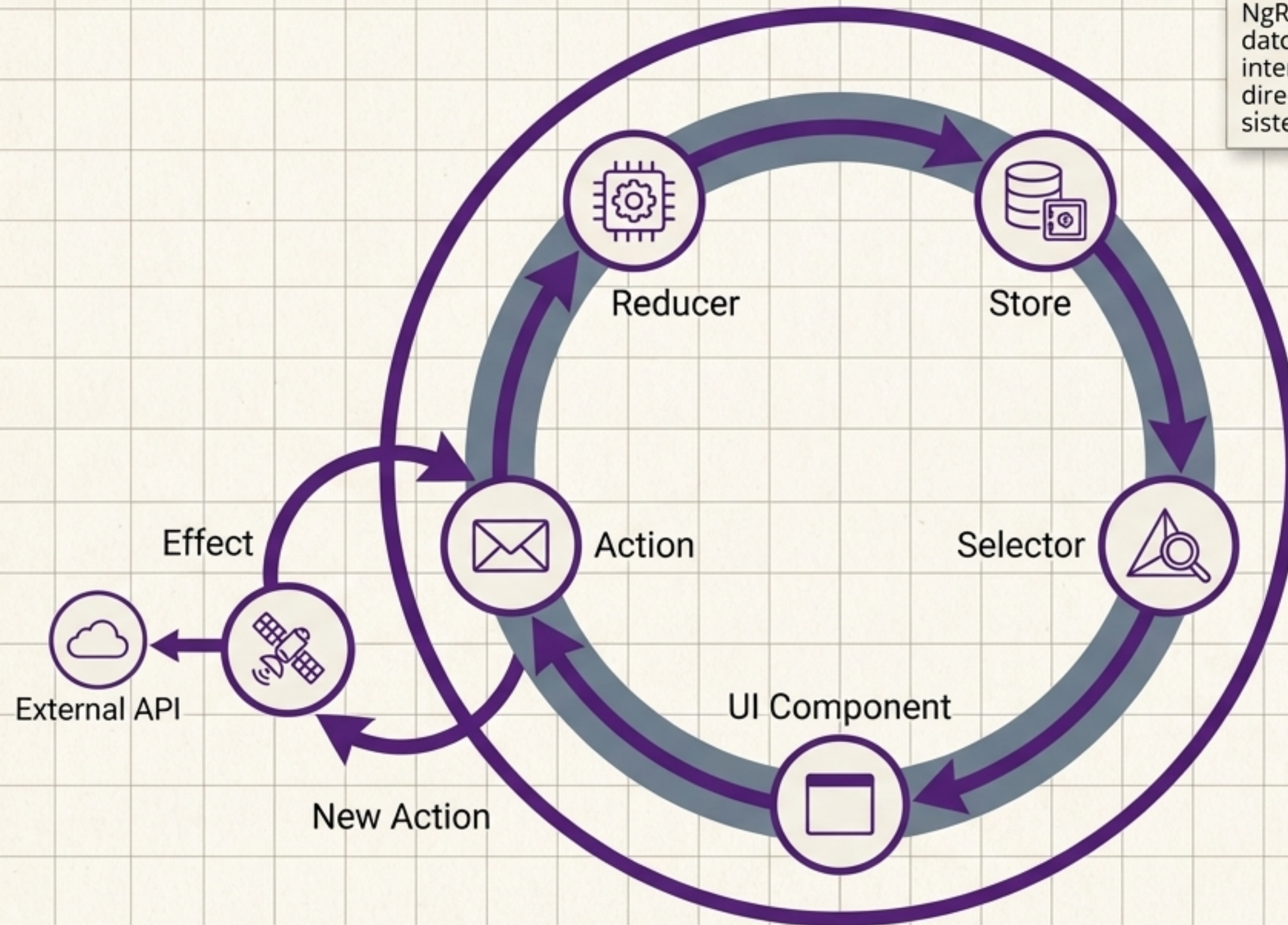
Cuando la aplicación crece, el uso exclusivo de BehaviorSubjects aislados genera flujos impredecibles. Múltiples fuentes modifican datos compartidos, haciendo imposible rastrear el origen de una mutación.

La Solución: NgRx



Basado en el [patrón Redux](#), NgRx centraliza la gestión del estado global. Transforma el caos de múltiples flujos reactivos en una única fuente de verdad, con mutaciones predecibles, auditables e inmutables.

El Circuito Inmutable: Flujo Unidireccional NgRx



NgRx impone una estricta direccionalidad. Los datos nunca fluyen hacia atrás. La interfaz de usuario nunca modifica el estado directamente; solo describe eventos que el sistema procesa en un ciclo cerrado.

La Fase de Escritura: Actions & Reducers



Actions

- Objetos simples que describen qué ocurrió en la aplicación (ej. "Botón de carga clickeado").
- Son los únicos detonantes legítimos capaces de iniciar una **transición** de estado.



Reducers

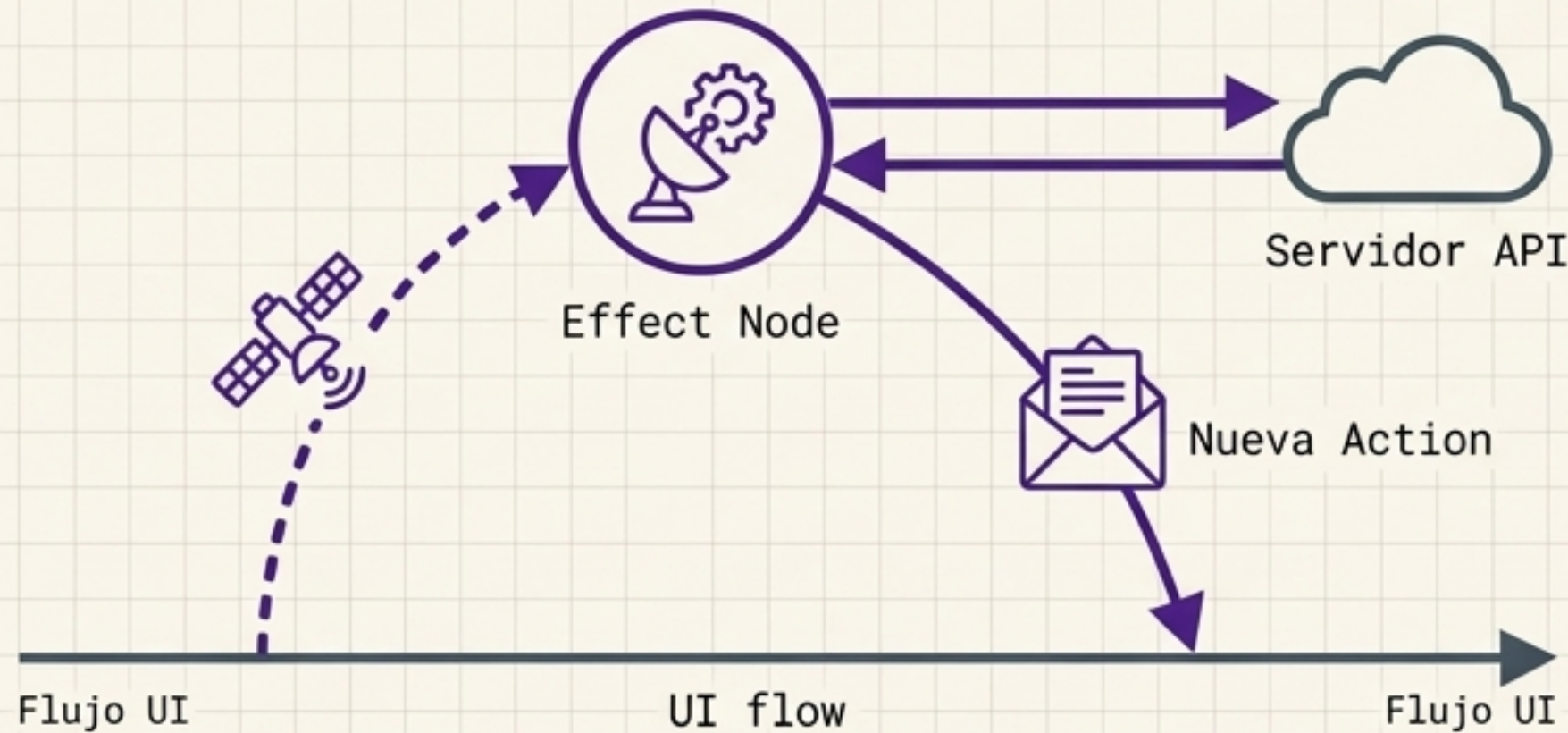
- Funciones puras que calculan la mutación matemática del estado.
- **Mecánica Estricta:** Reciben el estado actual y la acción disparada. Nunca mutan el objeto original; siempre devuelven un nuevo objeto de estado (Inmutabilidad).

La Fase de Lectura: Store & Selectors



Effects: La Órbita Asíncrona

Los Reducers tienen prohibido ejecutar código asíncrono. Los Effects asumen esta responsabilidad, aislando los "side effects" de la lógica pura.



1

Escucha

Interceptan Actions específicas disparadas por la UI (ej. "Solicitar Datos").

2

Ejecuta

Realizan operaciones asíncronas bloqueantes, como llamadas HTTP a una API o lecturas de base de datos externa.

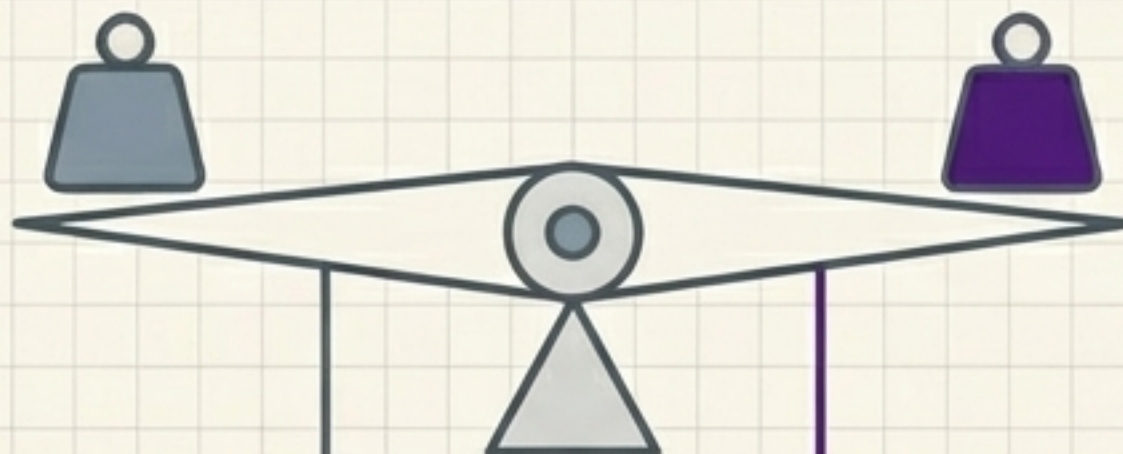
3

Responde

Disparan una nueva Action (ej. "Carga Exitosa" o 'Error') con el payload resultante, devolviendo el control al flujo unidireccional.

El Criterio Arquitectónico: ¿Cuándo justificar NgRx?

Mantener Estado Local



Implementar NgRx



Contexto:

Aplicaciones pequeñas o prototipado rápido.

Condiciones:

El estado vive y muere en un solo componente.

Veredicto:

El overhead de configuración de NgRx no justifica el valor aportado. **RxJS** puro es suficiente.

Contexto:

Estado compartido extensamente entre múltiples ramas de componentes.

Condiciones:

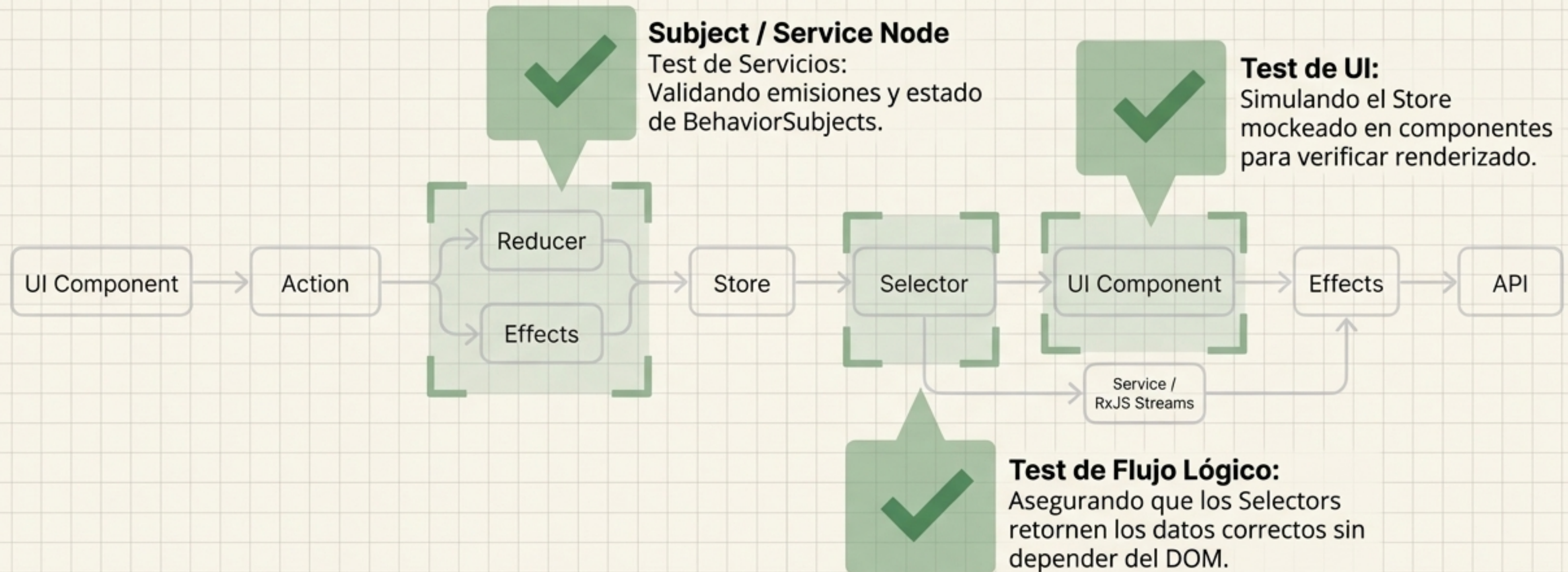
Flujos asíncronos complejos y múltiples fuentes modificando el mismo fragmento de estado simultáneamente.

Veredicto:

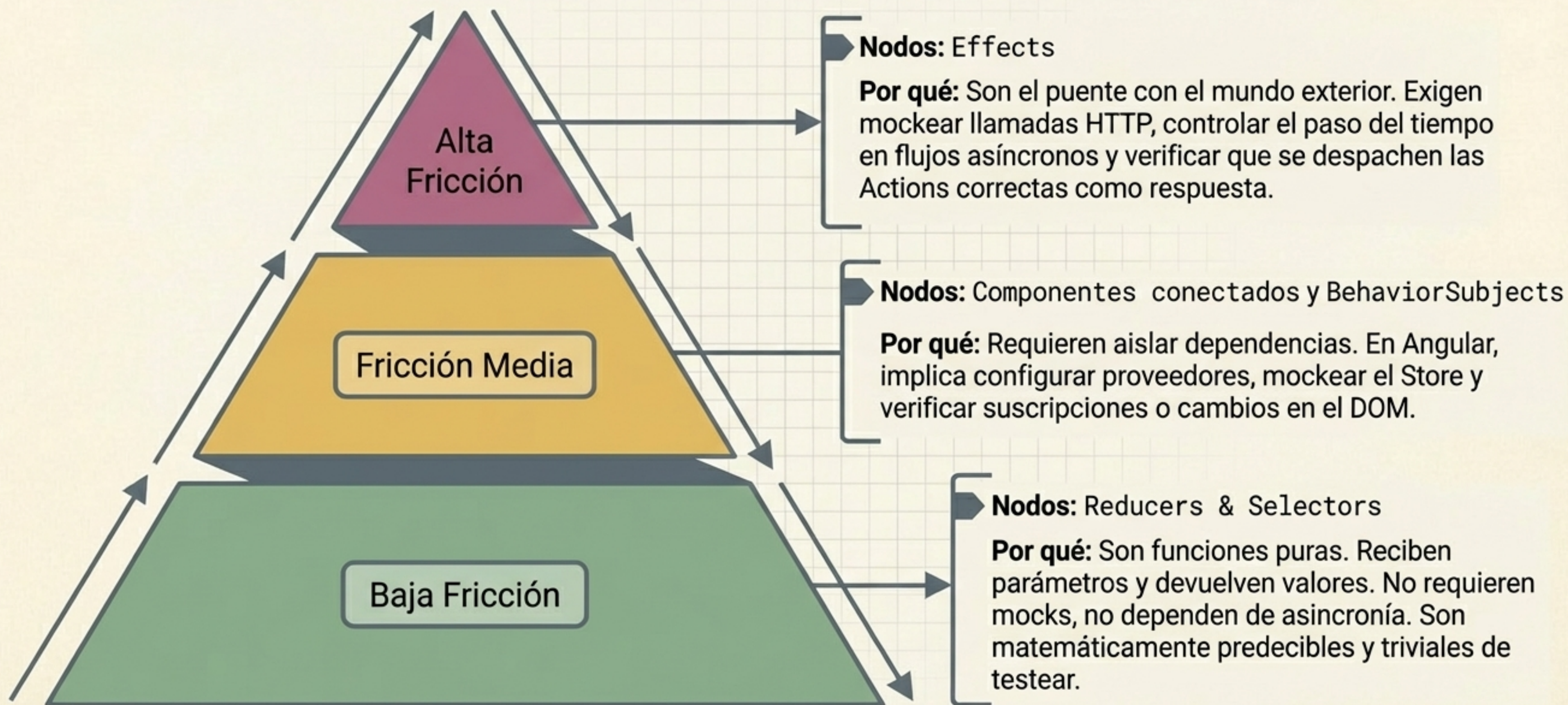
Se requiere previsibilidad absoluta y capacidades avanzadas de auditoría temporal (**Redux DevTools**).

Jest: La Capa de Garantía y Validación

Una arquitectura reactiva predecible exige una estrategia de testing rigurosa. Jest actúa como el framework definitivo, proporcionando ejecución rápida, matchers potentes y capacidades de simulación (mocks) para aislar cada pieza del circuito.



El Mapa de Fricción de Testing



Síntesis Arquitectónica (Cheat Sheet)

RxJS (Motor Reactivo)	NgRx (Gestión de Estado)	Jest (Validación)
Observable Stream de datos asíncronos de solo lectura.	Store Objeto inmutable, única fuente de verdad global.	Testing Puro Validación directa e instantánea de las salidas matemáticas de Reducers y Selectors.
BehaviorSubject Estado compartido con memoria; el valor actual es directamente accesible en cualquier momento.	Actions Descriptores planos de eventos pasados.	Mocks & Matchers Simulación de dependencias (HTTP, Store) para aislar rigurosamente Componentes y Effects.
	Reducers Funciones puras que calculan la mutación.	
	Selectors Filtros memoizados de extracción de datos.	
	Effects Gestores aislados de operaciones asíncronas.	